

Base de Dados

“SIESTÁGIOS PT2”

Gonçalo Sobral Nº 129850

Daniel Masqueiro Nº 129853

Rafael Silva Nº 129834

Turma PL03

2025/2026

Índice

1.	Introdução	2
2.	Automatismos	3
2.1.	Triggers	3
2.1.1.	T1	3
2.1.2.	T2	3
2.2.	Stored Procedures	3
2.2.1.	P1	3
2.2.2.	P2	3
2.3.	Funções	4
2.3.1.	F1	4
2.3.2.	F2	4
3.	Consultas	
3.1.	SQL-DML	4
3.1.1.	Q1	4
3.1.2.	Q2	4
3.1.3.	Q3	5
3.1.4.	Q4	5
3.1.5.	Q5	5
3.1.6.	Q6	5
3.2.	Views	6
3.2.1.	V1	6
3.2.2.	V2	6
4.	Conclusão	
4.1.	Síntese do Projeto	6
4.2.	Críticas	7
4.3.	Apreciação Final	7
4.4.	Erros	7
4.5.	Observações.....	7
5.	Anexos	9

1. Introdução

Este relatório é sobre o desenvolvimento da segunda parte do projeto “SIEstágios” para a gestão de estágios de uma escola profissional. Enquanto a primeira parte focava-se no desenho e implementação da estrutura de dados, esta segunda concentra-se na criação de mecanismos de automação, otimização e acesso de dados e a criação de um protótipo de aplicação web.

Esta parte serve para facilitar a gestão de estágios permitindo o registo de alunos e estágios, consulta de empresas disponíveis para estagiar e o lançamento e consulta de notas.

De acordo com o enunciado, serão desenvolvidos os seguintes três componentes principais:

Automatismos: Criação de triggers, stored procedures e funções. Estes serão úteis para a validação, manutenção e cálculos.

Consultas SQL-DML e Views: Criar um conjunto de consultas para apresentar algumas informações relevantes do sistema, como estatísticas sobre formadores e empresas. Foram desenvolvidas seis queries e duas Views.

Protótipo Web: Criação de uma aplicação web em PHP que contém três portais: Administrador, Formador e Aluno.

2. Automatismos

2.1 Triggers

2.1.1 T1 - foi criado o trigger **t1_classificacao_insert** (anexo 1) e **t1_classificacao_update** (anexo 2).

Objetivo: Impedir que ao inserir ou ao atualizar os registos do estágio, neste caso que a nota inserida pelo estagiário ao local de estágio esteja comprimida entre os valores inteiros de 1 a 5.

Funcionamento: O trigger é acionado antes de uma operação de **INSERT** ou **UPDATE** da tabela Estagio. Verifica se o campo **Classificacao** não é **NULL** e se está fora do intervalo [1, 5]. Se a validação falhar, lança uma mensagem de erro. Se a classificação for **NULL** (ainda não atribuída), o trigger permite a operação.

2.1.2 T2 - foi criado o trigger **t2_data_fim_data_inicio_insert** (anexo 3) e **t2_data_fim_data_inicio_update** (anexo 4).

Objetivo: Impedir que seja inserido/atualizado uma **data_inicio** posterior à **data_fim**, garantindo assim a consistência dos dados.

Funcionamento: O trigger é ativado **ANTES** de qualquer operação de **UPDATE** ou **INSERT** na tabela Estagio. É verificado se ambas as datas estão preenchidas. Compara as datas para garantir que **Data_Inicio** não é posterior à **Data_Fim**. Se a validação falhar, impede a atualização e retorna uma mensagem de erro. Permite **NULL** em qualquer das datas (para casos onde o estágio ainda não está completamente definido).

2.2 Stored Procedures

2.2.1 P1 - foi criado o stored procedure **p1_registar_estagios** (anexo 5)

Objetivo: Criar uma stored procedure que regista um novo estágio, verificando antes a existência de todas as entidades (aluno, formador, estabelecimento).

Funcionamento: Verifica a existência de cada um dos campos (aluno, formador, estabelecimento). Cada validação falhada retorna uma mensagem que indica qual campo não existe. Só após todas as validações é que o estágio é inserido. Após todas as validações, insere o registo do estágio na tabela com valores iniciais (todas as notas a 0 e classificação **NULL**).

2.2.2 P2 - foi criado o stored procedure **p2_next_estagio** (anexo 6)

Objetivo: Criar uma stored procedure que lista os estágios que se iniciam dentro de um número específico de dias a partir da data corrente.

Funcionamento: Cálculo de diferença de dias: Utiliza **DATEDIFF(e.Data_Inicio, CURDATE())** para calcular quantos dias faltam até o início. **BETWEEN 0 AND p_numero_dias** garante que só aparecem estágios futuros dentro do prazo. Depois ordena-se por data de início (do mais próximo ao mais distante).

2.3 Funções

2.3.1 F1 - foi criado a função **f1_calculo_media_estabelecimentos** (anexo 7)

Objetivo: Criar uma função que calcule a média das classificações dadas pelos estagiários a um estabelecimento específico num determinado ano letivo.

Funcionamento: Aplica três filtros simultâneos: filtra pelo ID da empresa; pelo ID do estabelecimento e pelo ano letivo pretendido. O **WHERE e.classificacao IS NOT NULL** garante que apenas estágios com classificação já atribuída são considerados no cálculo. Utiliza a função agregada **AVG()** para calcular a média de todas as classificações. A função **IFNULL(v_media, 0)** garante que, caso não existam estágios com classificação para os parâmetros fornecidos, a função retorna 0 em vez de **NULL**.

2.3.2 F2 - foi criado a função **f2_calculo_nota_final** (anexo 8)

Objetivo: Criar uma função que calcule a média ponderada da nota final de um estágio, permitindo atribuir pesos diferentes a cada componente de avaliação (nota da empresa, nota da escola, nota de procura e nota do relatório).

Funcionamento: Lê todas as notas do estágio especificado. Verifica se o estágio existe e tem pelo menos uma nota. Apenas considera notas que não sejam **NULL**. Ajusta automaticamente os pesos para somar 100% apenas com as notas disponíveis.

3. Consultas

3.1 SQL-DML

3.1.1 Q1 - Formadores com Mais de 1 Estágio (anexo 9)

Objetivo: Apresentar o nome do formador e o número total de estágios que supervisionou, apenas para formadores com mais de 1 estágio.

Explicação: Liga a tabela Formador à Utilizador para obter o nome. Liga-se aos estágios para contar quantos cada formador está a avaliar. **GROUP BY:** Agrupa por formador para calcular o total por pessoa. **HAVING:** Filtra apenas formadores com mais de 1 estágio.

3.1.2 Q2 - Empresas com Média de Notas ≥ 14 (anexo 10)

Objetivo: Apresentar o nome da empresa e a média das notas atribuídas aos alunos, apenas para empresas com média igual ou superior a 14.

Explicação: Múltiplos JOINS: Liga Empresa → Estabelecimento → Responsavel → Estagio. **WHERE:** Garante que apenas estágios com nota atribuída são considerados. **AVG(e.Nota_Empresa):** Calcula a média das notas dadas pela empresa. **ROUND(..., 2):** Arredonda o resultado para 2 casas decimais. **HAVING:** Filtra apenas empresas com média ≥ 14 . **GROUP BY:** Agrupa por empresa para calcular a média de cada uma.

3.1.3 Q3 - Empresas que Comercializam Produtos (anexo 11)

Objetivo: Apresentar o nome da empresa e o total de produtos comercializados, para empresas que comercializam pelo menos 1 produto.

Explicação: **JOIN com Estabelecimento_produtos:** Acede à tabela associativa entre estabelecimentos e produtos. **COUNT(DISTINCT ...):** Conta produtos únicos (evita duplicações). **GROUP BY:** Agrupa por empresa. **HAVING:** Filtra empresas com pelo menos 1 produto.

3.1.4 Q4 - Empresas com Estágios (anexo 12)

Objetivo: Apresentar o nome da empresa e o número total de estágios, apenas para empresas com pelo menos 1 estágio.

Explicação: Cadeia de **JOINS:** Empresa → Estabelecimento → Responsavel → Estagio. **COUNT(e.Estagio_ID):** Conta o número de estágios por empresa. **GROUP BY:** Agrupa resultados por empresa. **HAVING:** Filtra empresas com pelo menos 1 estágio. **ORDER BY:** Ordena por volume de estágios (descendente).

3.1.5 Q5 - Cursos com Turmas Acima da Média (anexo 13)

Objetivo: Apresentar o nome dos cursos que têm um número de turmas superior à média geral de turmas por curso.

Explicação: **Subquery:** Conta turmas por curso. **Subquery:** Calcula a média de turmas por curso. **HAVING:** Compara cada curso com a média geral. **GROUP BY:** Agrupa por curso para contar turmas. Só mostra cursos que excedem a média.

3.1.6 Q6 - Formadores com Média Superior à Global (anexo 14)

Objetivo: Apresentar o nome dos formadores, a média das notas dos estágios finais que orientaram e a média global de todas as notas de estágio finais, apenas para formadores cuja média individual seja superior à média global.

Explicação: **Cálculo da nota final:** Média aritmética das 4 componentes. **Subquery na SELECT:** Mostra a média global para comparação. **WHERE:** Garante que apenas estágios completos são considerados. **HAVING:** Filtra formadores acima da média. **ROUND(..., 2):** Formata os valores para 2 casas decimais.

3.2 Views

3.2.1 V1 - Detalhes dos Estágios por Formador (anexo 15)

Objetivo: Criar uma view que apresenta os detalhes dos estágios orientados pelos formadores, incluindo o nome do formador, o número de estágios que supervisionou, a média das notas finais desses estágios e a média geral de todas as notas finais de estágio.

Funcionamento: A view usa joins entre as tabelas Formador, Utilizador e Estagio para obter o nome completo de cada formador e os seus estágios associados. Utiliza **COUNT(e.aluno_id)** para contar o total de estágios orientados e **AVG(e.nota_final)** para calcular a média das notas finais, arredondando ambos os valores para duas casas decimais. Uma subquery em **SELECT** calcula independentemente a média global de todas as notas finais, permitindo assim comparar cada formador pelo seu desempenho geral. Os resultados são agrupados por nome do formador através de **GROUP BY**, consolidando toda a informação de cada um numa única linha. Isto torna as consultas complexas acessíveis através de um **SELECT** sobre a view.

3.2.2 V2 - Média de Notas por Empresa e Curso (anexo 16)

Objetivo: Criar uma view que mostra a média das notas finais de estágio de cada empresa, agrupadas por empresa e curso.

Funcionamento: A view usa vários joins que seguem dois caminhos simultâneos: o caminho para a empresa (Estagio → Estabelecimento → Empresa) para obter o nome da firma, e o caminho “académico” (Estagio → Aluno → Turma → Curso) para identificar a designação do curso. A função **AVG(e.nota_final)** calcula a média das notas finais, arredondada para 2 casas decimais.

4. Conclusão

4.1 Síntese do Projeto

Com esta segunda parte do projeto "SIEstágios" focamo-nos na implementação de mecanismos de automação, otimização e acesso à base de dados, bem como no desenvolvimento de um protótipo web funcional para a demonstração do sistema a funcionar.

Automatismos implementados:

2 Triggers (T1 e T2) que garantem a integridade e validação de dados críticos, impedindo a inserção de classificações inválidas e datas inconsistentes. 2 Stored Procedures (P1 e P2) que ajudam com operações complexas de registo de estágios e consulta de estágios próximos do início. 2 Funções (F1 e F2) que realizam cálculos essenciais para a avaliação de estabelecimentos e notas finais ponderadas

Consultas e Views:

6 Queries SQL (Q1 a Q6) que extraem informações relevantes sobre formadores, empresas, cursos, etc. 2 Views (V1 e V2) que consolidam informações complexas e facilitam certas análises.

Protótipo Web:

3 Portais distintos (Administrador, Aluno e Formador) que permitem gestão completa do ciclo de vida dos estágios. Interface intuitiva que demonstra a aplicação prática da base de dados em ambiente real.

4.2 Críticas

Não existe distinção clara se um aluno pode estagiar no mesmo estabelecimento várias vezes. A estrutura da base de dados permite esta situação através de chaves compostas que incluem `aluno_id`, mas não existe validação que impeça ou permita explicitamente múltiplos estágios do mesmo aluno no mesmo local em períodos diferentes.

O T2 não valida se as datas de estágio respeitam o calendário académico ou períodos letivos definidos.

Formadores novos precisam ser inseridos diretamente na base de dados via SQL, já que não foi implementada uma maneira de inserir pelo Protótipo Web.

4.3 Apreciação Final

O desenvolvimento da Parte 2 do projeto revelou-se um exercício valioso na compreensão e aplicação de conceitos avançados de bases de dados relacionais. A transição de um modelo conceitual (Parte 1) para uma implementação funcional com automatismos e interface web permitiu consolidar conhecimentos teóricos e práticos.

4.4 Erros

Ao longo do projeto deparamo-nos com alguns erros maioritariamente na fase do desenvolvimento do Protótipo Web.

1. **Erro SQLSTATE[2300] (Anexo 17)** - este erro apareceu-nos já que ao criar um novo aluno era inserido com ID 0 automaticamente para resolver este problema tivemos de mexer no SQL, mas isto levou-nos para outro erro;
2. **Erro MySQL #1834 (Anexo 18)** - ao tentar colocar o `utilizador_id` como **AUTO_INCREMENT** deparamo-nos com este erro. Para resolver este erro tivemos de desabilitar as FKs da base de dados alterar `utilizador_id` para **AI**. Resolvendo assim ambos os erros 1 e 2.

4.5 Observações

1. Foi desenvolvido um módulo específico na aplicação web ("Relatórios de Gestão") para a visualização das *Queries* e *Views* desenvolvidas na Parte 2 do projeto. Em vez de executar estas consultas apenas no SGBD, integrámos as mesmas na interface do Administrador através de código PHP que estabelece ligação à base de dados, executa as queries SQL e apresenta os resultados de forma formatada em tabelas HTML. Esta implementação permite uma leitura imediata e intuitiva dos indicadores de gestão (ex: formadores com mais estágios, médias por empresa, etc.) diretamente no portal web, sem necessidade de acesso direto ao phpMyAdmin.

2. Para mitigar erros de inserção manual e conflitos de chaves primárias (*Duplicate Entry*), a estrutura da base de dados foi alterada para utilizar a propriedade

AUTO_INCREMENT na coluna **utilizador_id**. No código PHP, foi implementada a captura automática deste ID (via método **lastInsertId()**), garantindo a correta propagação da chave estrangeira para a tabela subclasse (**Aluno**) sem intervenção humana.

3. Foi também implementado um sistema completo na gestão de alunos no painel do Administrador. Esta funcionalidade permite não apenas criar, mas também **editar e remover** registros de alunos. Isto foi essencial para permitir a correção de erros comuns (ex: gralhas no nome ou turma errada) diretamente pela interface, sem necessidade de recorrer a comandos SQL diretos na base de dados.

4. Melhorámos a lógica da atribuição de notas através da função SQL **f2_calculo_nota_final**. A função valida que todas as componentes de avaliação (Empresa, Escola, Relatório e Procura) existam antes de calcular a média final. Desta forma, o sistema garante que a nota final só é gerada quando o processo de avaliação está integralmente concluído.

5. Relativamente ao requisito do enunciado sobre o registo de estágios, que indica *"apenas deve ser inserido o Estabelecimento, Empresa, Data de início, o Aluno, e o Formador"*, optámos por incluir também a **Data de Fim** no momento do registo. Esta decisão foi tomada por duas razões principais: 1. Ao solicitar ambas as datas no formulário, permitimos que o Trigger T2 (**t2_data_fim_data_inicio_insert**) valide a consistência do período de estágio logo no momento da criação do registo, impedindo a entrada de dados inconsistentes na base de dados. 2. Na prática, quando um estágio é planeado e registado pela instituição, ambas as datas (início e fim) já são conhecidas e definidas contratualmente entre a escola, o aluno e a empresa. Registrar esta informação completa desde o início facilita o planeamento do estágio e das avaliações por parte dos formadores.

6. Para resolver alguns Fatal Errors, melhorar a experiência do utilizador e evitar a apresentação de erros técnicos (fatal errors) da base de dados, foram implementadas validações no lado do servidor antes das operações de inserção e atualização. Esta abordagem foi desenvolvida seguindo dois tutoriais *"PHP MySQL Error Handling: Beginner's Guide"* e *"PHP Try Catch Error Handling"* de uns canais de YouTube, que demonstra como tratar estes erros. As validações implementadas trabalham em conjunto com os triggers da base de dados.

7. Identificámos alguns erros críticos (Fatal Errors) na parte dos PHPs que ainda persistem no sistema. Apesar dos nossos esforços para os corrigir, houve alguns que só encontramos perto da entrega do projeto e não conseguimos corrigir a tempo.

Anexos

Anexo 1 (t1_classificacao_insert) e 2 (t1_classificacao_update)

```
BEGIN
  IF NEW.classificacao IS NOT NULL AND (NEW.classificacao < 1 OR NEW.classificacao >
5) THEN
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Erro: A classificacao esta fora
do intervalo (1 a 5).';
  END IF;
END
```

Anexo 3 (t2_data_fim_data_inicio_insert) e 4 (t2_data_fim_data_inicio_update)

```
BEGIN
  IF NEW.data_inicio > NEW.data_fim THEN
    SIGNAL SQLSTATE '45000'
    SET MESSAGE_TEXT = 'Erro: A data de inicio do estagio não pode ser posterior a
data de fim.';
  END IF;
END
```

Anexo 5 (p1_registar_estagios)

```
BEGIN
  DECLARE v_aluno_existe INT DEFAULT 0;
  DECLARE v_formador_existe INT DEFAULT 0;
  DECLARE v_estabelecimento_existe INT DEFAULT 0;

  SELECT COUNT(*) INTO v_aluno_existe FROM Aluno WHERE utilizador_id =
p_aluno_id;
  SELECT COUNT(*) INTO v_formador_existe FROM Formador WHERE utilizador_id =
p_formador_id;
  SELECT COUNT(*) INTO v_estabelecimento_existe FROM Estabelecimento WHERE
empresa_id = p_empresa_id AND estabelecimento_id = p_estabelecimento_id;

  IF v_aluno_existe = 0 THEN
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Erro: Aluno não existe na base
de dados';
  END IF;
  IF v_formador_existe = 0 THEN
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Erro: Formador não existe na
base de dados';
  END IF;
  IF v_estabelecimento_existe = 0 THEN
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Erro: Estabelecimento não
existe na base de dados';
  END IF;
```

```

INSERT INTO Estagio (
    aluno_id, formador_id,
    estabelecimento_empresa_id, estabelecimento_id,
    data_inicio, data_fim,
    nota_escola, nota_relatorio, nota_procura, nota_empresa, classificacao
) VALUES (
    p_aluno_id, p_formador_id, p_empresa_id, p_estabelecimento_id,
    p_data_inicio, p_data_fim,
    0, 0, 0, 0, NULL
);
END

```

Anexo 6 (p2_next_estagio)

```

BEGIN
    SELECT
        u_aluno.nome AS Nome_Aluno,
        u_formador.nome AS Nome_Formador,
        emp.firma AS Nome_Empresa,
        est.nome_comercial AS Nome_Estabelecimento,
        e.data_inicio,
        DATEDIFF(e.data_inicio, CURDATE()) AS Dias_Ate_Inicio
    FROM Estagio e
    INNER JOIN Aluno a ON e.aluno_id = a.utilizador_id
    INNER JOIN Utilizador u_aluno ON a.utilizador_id = u_aluno.utilizador_id
    INNER JOIN Formador f ON e.formador_id = f.utilizador_id
    INNER JOIN Utilizador u_formador ON f.utilizador_id = u_formador.utilizador_id
    INNER JOIN Estabelecimento est ON e.estabelecimento_empresa_id = est.empresa_id
        AND e.estabelecimento_id = est.estabelecimento_id
    INNER JOIN Empresa emp ON est.empresa_id = emp.empresa_id
    WHERE DATEDIFF(e.data_inicio, CURDATE()) BETWEEN 0 AND p_numero_dias
    ORDER BY e.data_inicio ASC;
END

```

Anexo 7 (f1_calculo_media_estabelecimentos)

```

BEGIN
    DECLARE v_media DECIMAL(10, 1);

    SELECT AVG(e.classificacao) INTO v_media
    FROM Estagio e
    INNER JOIN Aluno a ON e.aluno_id = a.utilizador_id
    INNER JOIN Turma t ON a.turma_id = t.turma_id
    WHERE e.estabelecimento_empresa_id = p_empresa_id
        AND e.estabelecimento_id = p_estabelecimento_id
        AND t.ano = p_ano
        AND e.classificacao IS NOT NULL;

    RETURN IFNULL(v_media, 0);
END

```

Anexo 8 (f2_calculo_nota_final)

```
BEGIN
  DECLARE v_nota_empresa FLOAT;
  DECLARE v_nota_relatorio FLOAT;
  DECLARE v_nota_escola FLOAT;
  DECLARE v_nota_procura FLOAT;
  DECLARE v_media DECIMAL(10, 2);

  SELECT nota_empresa, nota_relatorio, nota_escola, nota_procura
  INTO v_nota_empresa, v_nota_relatorio, v_nota_escola, v_nota_procura
  FROM Estagio
  WHERE estabelecimento_empresa_id = p_empresa_id
    AND estabelecimento_id = p_estabelecimento_id
    AND aluno_id = p_aluno_id;

  IF v_nota_empresa IS NULL OR
    v_nota_relatorio IS NULL OR
    v_nota_escola IS NULL OR
    v_nota_procura IS NULL THEN
    RETURN NULL;
  END IF;

  IF ABS((p_peso_empresa + p_peso_escola + p_peso_relatorio + p_peso_procura) - 1.0)
  > 0.01 THEN
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Erro: A soma dos pesos deve
  ser 1';
  END IF;

  SET v_media = (v_nota_empresa * p_peso_empresa) +
    (v_nota_relatorio * p_peso_relatorio) +
    (v_nota_escola * p_peso_escola) +
    (v_nota_procura * p_peso_procura);

  RETURN v_media;
END
```

Anexo 9 (Q1)

```
SELECT u.nome,
COUNT(e.aluno_id) as total FROM Formador f
JOIN Utilizador u ON f.utilizador_id = u.utilizador_id
JOIN Estagio e
ON f.utilizador_id = e.formador_id
GROUP BY u.nome HAVING total > 1
```

Anexo 10 (Q2)

```
SELECT emp.firma,
ROUND(AVG(e.nota_empresa), 2) as media FROM Empresa emp
JOIN Estabelecimento est ON emp.empresa_id = est.empresa_id
JOIN Estagio e ON est.empresa_id = e.estabelecimento_empresa_id AND
est.estabelecimento_id = e.estabelecimento_id
GROUP BY emp.firma HAVING media >= 14
```

Anexo 11 (Q3)

```
SELECT emp.firma,
COUNT(c.produto_id) as total FROM Empresa emp
JOIN Estabelecimento est ON emp.empresa_id = est.empresa_id
JOIN Comercializa c ON est.empresa_id = c.estabelecimento_empresa_id AND
est.estabelecimento_id = c.estabelecimento_id
GROUP BY emp.firma HAVING total >= 1
```

Anexo 12 (Q4)

```
SELECT emp.firma,
COUNT(e.aluno_id) as total FROM Empresa emp
JOIN Estabelecimento est ON emp.empresa_id = est.empresa_id
JOIN Estagio e ON est.empresa_id = e.estabelecimento_empresa_id AND
est.estabelecimento_id = e.estabelecimento_id
GROUP BY emp.firma HAVING total >= 1
```

Anexo 13 (Q5)

```
SELECT c.designacao,
COUNT(t.turma_id) as total FROM Curso c
JOIN Turma t ON c.curso_id = t.curso_id
GROUP BY c.designacao HAVING total > (SELECT AVG(contagem)
FROM (SELECT COUNT(turma_id) as contagem FROM Turma GROUP BY curso_id) as
sub)");
```

Anexo 14 (Q6)

```
SELECT u.nome,
ROUND(AVG(e.nota_final), 2) as media FROM Formador f
JOIN Utilizador u ON f.utilizador_id = u.utilizador_id
JOIN Estagio e ON f.utilizador_id = e.formador_id
GROUP BY u.nome HAVING media > (SELECT AVG(nota_final) FROM Estagio)
```

Anexo 15 (v1_detalhes_formadores)

```
select      `u`.`nome`      AS      `Nome_Formador`,count(`e`.`aluno_id`)      AS
`Total_Estagios`,round(avg(`e`.`nota_final`),2)      AS      `Media_Formador`,(select
round(avg(`estagios_parte2`.`estagio`.`nota_final`),2) from `estagios_parte2`.`estagio`) AS
`Media_Global_Todos`      from      ((`estagios_parte2`.`formador`      `f`      join
`estagios_parte2`.`utilizador`      `u`      on(`f`.`utilizador_id`      =      `u`.`utilizador_id`)) join
`estagios_parte2`.`estagio` `e` on(`f`.`utilizador_id` = `e`.`formador_id`)) group by `u`.`nome`
```

Anexo 16 (v2_media_empresa_curso)

```
select      `emp`.`firma`      AS      `Nome_Empresa`,`c`.`designacao`      AS
`Nome_Curso`,round(avg(`e`.`nota_final`),2)      AS      `Media_Notas`      from
((((`estagios_parte2`.`estagio`      `e`      join      `estagios_parte2`.`estabelecimento`      `est`
on(`e`.`estabelecimento_empresa_id` = `est`.`empresa_id` and `e`.`estabelecimento_id` =
`est`.`estabelecimento_id`))) join `estagios_parte2`.`empresa`      `emp`      on(`est`.`empresa_id` =
`emp`.`empresa_id`))      join      `estagios_parte2`.`aluno`      `a`      on(`e`.`aluno_id` =
`a`.`utilizador_id`)) join `estagios_parte2`.`turma`      `t`      on(`a`.`turma_id` = `t`.`turma_id`)) join
`estagios_parte2`.`curso`      `c`      on(`t`.`curso_id` = `c`.`curso_id`))      group      by
`emp`.`firma`,`c`.`designacao`
```

Anexo 17 (Erro - SQLSTATE[23000]

Erro: SQLSTATE[23000]: Integrity constraint violation: 1062 Duplicate entry '0' for key 'PRIMARY'

Anexo 18 (Erro - #1834)

Erro

Comando SQL: [Copiar](#)

ALTER TABLE utilizador MODIFY utilizador_id INT(11) NOT NULL AUTO_INCREMENT;

Mensagens do MySQL: 

#1834 - Cannot delete rows from table which is parent in a foreign key constraint 'fk_administrativo_utilizador' of table 'administrativo'